Data Mining & Informatics Lecture #2

Amin Noroozi
University of Wolverhampton
a.noroozifakhabi@wlv.ac.uk
https://www.linkedin.com/in/amin-n-148350218/

Plan

- **Workshop plan**

- There will be warmup Python exercises in all workshops
- We will have an extra online tutorial session each week starting from the next week. The date and time will be announced on Canvas. Attendance is not mandatory

NOTE: Besides the main materials in each workshop, there will be extra exercises each week to help students improve their data mining skills

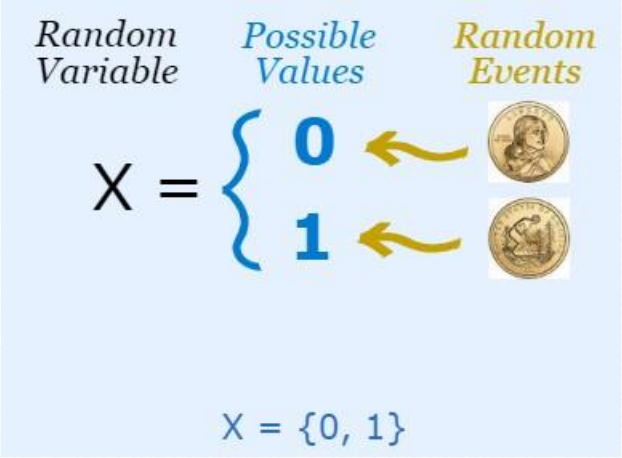
- Week #1 => Pandas
- Week #2 => Numpy
- Week #3 => Data visualisation

Statistics basics

Statistics

- **Random variable**

- **Random variable**: A Random variable is a variable containing a set of possible outcomes from a random experiment. It could be described using a function (distribution) that maps each outcome to a value in a probability space.
- For example, For instance, if X is used to denote the outcome of a coin toss ("the experiment"), then the probability distribution of X would take the value 0.5 (1 in 2 or 1/2) for X = heads, and 0.5 for X = tails (assuming that the coin is fair)



Statistics

- **Sample space and event**

Example: Throw a die once

Random Variable **X** = "The score shown on the top face".

X could be 1, 2, 3, 4, 5 or 6

So the Sample Space is {1, 2, 3, 4, 5, 6}

Statistics

- **Variable types**

NOTE: We treat dataset columns as random variables
NOTE: The data that a variable holds could be divided into two categories:

Quantitative data: which represents amounts.
Categorical data: which represents groupings.

NOTE: A variable that contains quantitative data is called a **quantitative variable**; a variable that contains categorical data is called a **categorical variable**. Each of these types of variables can be broken down into further types.

Statistics

- **Quantitative variable**

- When you collect quantitative data, the numbers you record represent real amounts that can be added, subtracted, divided, etc. There are two types of quantitative variables: **discrete** and **continuous**.

Discrete vs **Continuous** variables

Type of variable	What does the data represent	Examples
Discrete variables (aka integer variables)	Counts of individual items or values.	- Number of students in a class - Number of different tree species in a forest
Continuous variables (aka ratio variables)	Measurements of continuous or non-finite values.	- Distance - Volume - Age

Statistics

- **Categorical variable**

- Categorical variables represent groupings of some kind. They are sometimes recorded as numbers, but the numbers represent categories rather than actual amounts of things. There are three types of categorical variables: **binary**, **nominal**, and **ordinal** variables.

Binary vs **Nominal** vs **Ordinal** variables

Type of variable	What does the data represent	Examples
Binary variables (aka dichotomous variables)	Counts of individual items or values.	- Heads/tails in a coin flip - Win/lose in a football game
Nominal variables	Measurements of continuous or non-finite values.	- Species names - Colors - Brands
Ordinal variables	Groups that are ranked in a specific order.	- Finishing place in a race - Rating scale responses in a survey

Statistics

- **Categorical variable**

NOTE: Using discretization, we can turn a quantitative variable into a categorical variable. For example, if the variable contains age data ranging from 0 to 100, then we can divide the data into three categories: young (0-45), middle-aged (45-65), and old (65-100)

NOTE: Note that sometimes a variable can work as more than one type! An ordinal variable can also be used as a quantitative variable if the scale is numeric and doesn't need to be kept as discrete integers. For example, star ratings on product reviews are ordinal (1 to 5 stars), but the average star rating is quantitative.

Statistics

- **Independent, Dependent, and Control variables**

- Experiments are usually designed to find out what effect one variable has on another. For example, the effect of salt addition on plant growth.

- You manipulate the **independent variable** (the one you think might be the cause) and then measure the **dependent variable** (the one you think might be the effect) to find out what this effect might be.

- You will probably also have variables that you hold constant (**control variables**) in order to focus on your experimental treatment.

Statistics

- **Independent, Dependent, and Control variables**

Independent vs **Dependent** vs **Control** variables

Type of variable	What does the data represent	Examples
Independent variables (aka treatment or target variables)	Variables you manipulate in order to affect the outcome of an experiment.	The amount of salt added to each plant's water.
Dependent variables (aka response or input variables)	Variables that represent the outcome of the experiment.	Any measurement of plant health and growth: in this case, plant height and wilting.
Control variables	Variables that are held constant throughout the experiment.	The temperature and light in the room the plants are kept in, and the volume of water given to each plant.

Statistics

- **Descriptive statistics – central tendency - mean**

- The arithmetic mean (or simply mean) of a list of numbers, is the sum of all of the numbers divided by the number of numbers

$$\bar{x} = \frac{1}{n}\left(\sum_{i=1}^{n} x_i\right) = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

- For example, the arithmetic mean of five values: 4, 36, 45, 50, 75 is:

$$\frac{4 + 36 + 45 + 50 + 75}{5} = \frac{210}{5} = 42.$$

Statistics

- **Descriptive statistics – central tendency - median**

- The median of a finite list of numbers is the "middle" number, when those numbers are listed in order from smallest to greatest. Formally, a median of a population is any value such that at least half of the population is less than or equal to the proposed median and at least half is greater than or equal to the proposed median

$$\text{if } n \text{ is odd, } \mathrm{median}(x) = x_{(n+1)/2}$$

$$\text{if } n \text{ is even, } \mathrm{median}(x) = \frac{x_{(n/2)} + x_{((n/2)+1)}}{2}$$

- For example,

1, 3, 3, **6**, 7, 8, 9

Median = 6

1, 2, 3, **4**, **5**, 6, 8, 9

Median = (4 + 5) ÷ 2

= 4.5

Statistics

- **Descriptive statistics – central tendency - mode**
- The mode of a sample is the element that occurs most often in the collection. For example, the mode of the sample [1, 3, 6, 6, 6, 6, 7, 7, 12, 12, 17] is 6.
- Given the list of data [1, 1, 2, 4, 4] its mode is not unique. A dataset, in such a case, is said to be bimodal, while a set with more than two modes may be described as multimodal.
- **NOTE**: We can use mean for quantitative data, mode and median for ordinal categorical data, and mode for nominal categorical data

Statistics

- **Descriptive statistics – variability – standard deviation and variance**
- The standard deviation is the average amount of variability in your dataset. It tells you, on average, how far each score lies from the mean. The larger the standard deviation, the more variable the data set is. Variance is calculated by squaring the standard deviation

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

σ = population standard deviation

N = the size of the population

x_i = each value from the population

μ = the population mean

Statistics

- **Descriptive statistics – variability – standard deviation and variance**

Example:
Find the standard deviation and variance for the following list [15, 3, 12, 0, 24, 3]

Raw data	Deviation from mean	Squared deviation
15	15 – 9.5 = 5.5	30.25
3	3 – 9.5 = -6.5	42.25
12	12 – 9.5 = 2.5	6.25
0	0 – 9.5 = -9.5	90.25
24	24 – 9.5 = 14.5	210.25
3	3 – 9.5 = -6.5	42.25
M = 9.5	Sum = 0	Sum of squares = 421.5

Variance = 421.5/5 = 84.3

Standard deviation = √84.3 = 9.18

Statistics

- **NOTE**
- You can use the statistics or numpy module for calculating mean, mode, median, and standard deviation.

Example:

Find mean, mode, median, and standard deviation of the following list [1,1,1,2,3,4,4,5,5]

```
import statistics as st
X=[1,1,1,2,3,4,4,5,5]
print(st.mode(X))
print(st.mean(X))
print(st.median(X))
print(st.stdev(X))
```

Statistics

- **Descriptive statistics – Frequency distribution**

- A data set is made up of a distribution of values, or scores. In tables or graphs, you can summarise the frequency of every possible value of a variable in numbers or percentages.

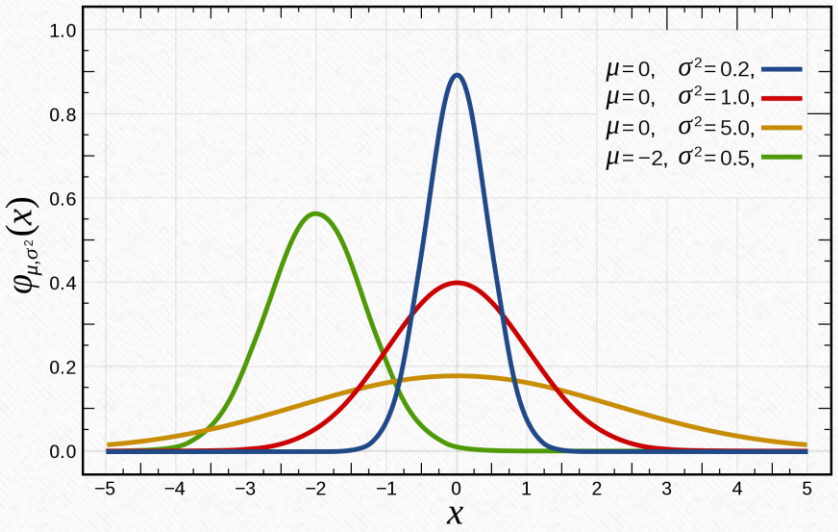
Gender	Frequency
Male	182
Female	235
Other	35

- Use the following syntaxt to show the frequency table of column ‘education’ in the adult dataset

```
data['education'].value_counts()
```

Statistics

- **Descriptive statistics – Normal distribution**



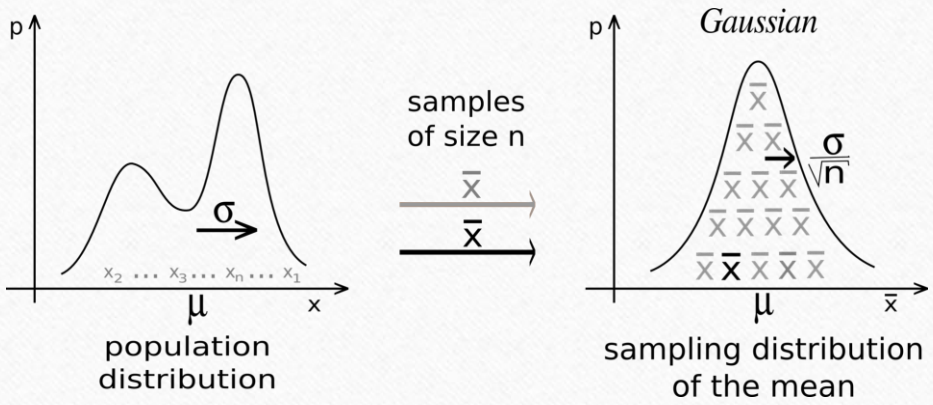
- Normal distributions are important in statistics and are often used in the natural and social sciences to represent real-valued random variables whose distributions are not known

Statistics

- **Descriptive statistics – Normal distribution**

- Their importance is partly due to the central limit theorem. It states that, in many situations, when independent random variables are summed up, their properly normalized sum tends toward a normal distribution even if the original variables themselves are not normally distributed.

$$\bar{X}_n \equiv \frac{X_1 + \cdots + X_n}{n}$$



Statistics

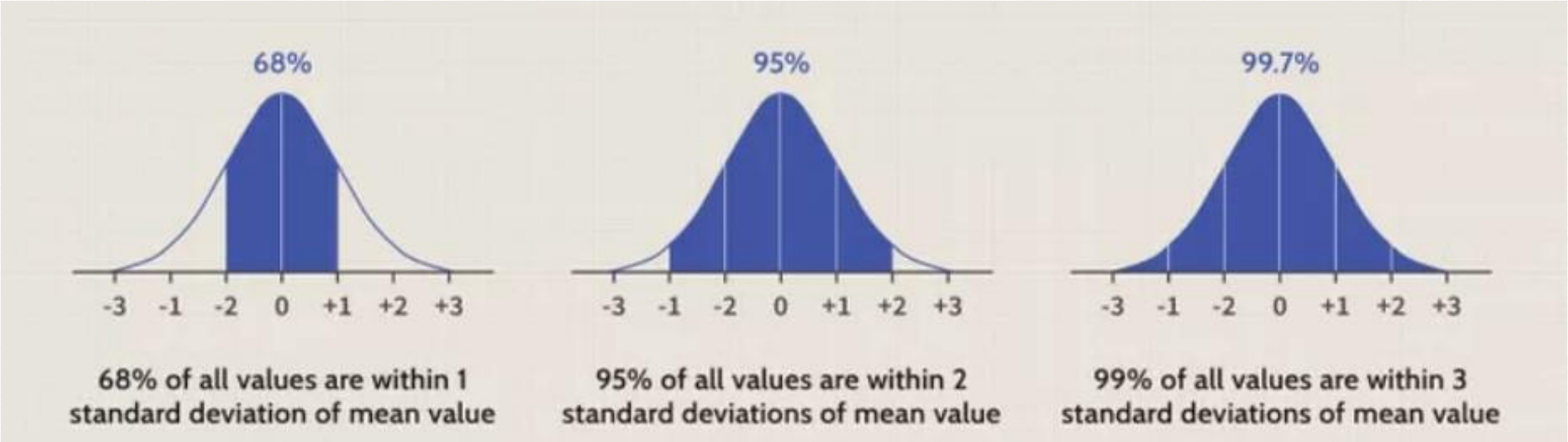
- **Descriptive statistics – Normal distribution**

NOTE: Mean (average), median (midpoint), and mode (most frequent observation) in a normal distribution are all equal to one another. Moreover, these values all represent the peak, or highest point, of the distribution. The distribution then falls symmetrically around the mean, the width of which is defined by the standard deviation.

NOTE: For all normal distributions, 68.2% of the observations will appear within plus or minus one standard deviation of the mean; 95.4% of the observations will fall within +/- two standard deviations; and 99.7% within +/- three standard deviations. This fact is sometimes referred to as the "empirical rule," a heuristic that describes where most of the data in a normal distribution will appear

Statistics

- **Descriptive statistics – Normal distribution**



Data preprocessing

Data preprocessing

- **Introduction**
- Before using datasets for analysis, we first need to preprocess them.
- There are different preprocessing steps depending on the experience of the data scientist analyzing the data.
- However, there are some common preprocessing steps that everyone needs to do before analysing datasets including viewing, visualising, cleaning, and encoding categorical data.

Data preprocessing

- **Viewing – head()**
- The head() method returns the headers and a specified number of rows, starting from the top.

Example (adult dataset)

```
import pandas as pd
df = data.copy()
print(df.head(10))
```

NOTE: You can use the set_option method to specify the maximum rows and columns. This method is especially helpful when working with big datasets

```
import pandas as pd
pd.set_option('display.max_rows', 100)
pd.set_option('display.max_columns', 100)
print(df.head(100))
```

Data preprocessing

- **Viewing – tail()**
- The tail() method returns the headers and a specified number of rows, starting from the bottom.

Example

```
import pandas as pd
print(df.tail())
```

Data preprocessing

- **Viewing – info()**
- The DataFrames object has a method called info(), that gives you more information about the data set

Example

```
import pandas as pd
print(df.info())
```

Data preprocessing

- **Cleaning**
- Data cleaning means fixing bad data in your data set.

- Bad data could be:

- Empty cells
- Wrong data
- Duplicates

- For this part we use an online dataset available at the following link:

https://www.w3schools.com/python/pandas/trypandas.asp?filename=demo_pandas_csv

Data preprocessing

- **Cleaning – Empty cells**
- Using the dropna() method we can drop rows with empty cells.

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.dropna().info())
```

- The fillna() method allows us to replace empty cells with a value

```
import pandas as pd
pd.set_option('display.max_rows', 170)
pd.set_option('display.max_columns', 100)
df = pd.read_csv('data.csv')
df["Calories"].fillna(130, inplace = True)
print(df.head(170))
```

Data preprocessing

- **Cleaning – Empty cells**
- A common way to replace empty cells, is to calculate the mean, median or mode value of the column. Pandas uses the mean() median() and mode() methods to calculate the respective values for a specified column.

```
import pandas as pd
pd.set_option('display.max_rows', 170)
pd.set_option('display.max_columns', 100)
df = pd.read_csv('data.csv')
x = df["Calories"].mean()
df["Calories"].fillna(x, inplace = True)
print(df.head(170))
```

Data preprocessing

- **Cleaning – Empty cells**
- We can also use interpolation to fill empty cells using the pandas interpolate method.

```
import pandas as pd
pd.set_option('display.max_rows', 170)
pd.set_option('display.max_columns', 100)
df = pd.read_csv('data.csv')
df.loc[:,"Calories"].interpolate(method="linear",inplace=True)
print(df.head(170))
```

Data preprocessing

- **Cleaning – Wrong data**
- Pandas has a to_datetime() method to correct wrong data formats

```
import pandas as pd
df = pd.read_csv('data.csv')
df['Date'] = pd.to_datetime(df['Date’])
```

25	60	'2020/12/25'	102	126	334.5
26	60	20201226	100	120	250.0
27	60	'2020/12/27'	92	118	241.0

25	60	'2020/12/25'	102	126	334.5
26	60	'2020/12/26'	100	120	250.0
27	60	'2020/12/27'	92	118	241.0

NOTE: You usually have to drop the wrong rows for other types of wrong data.

Data preprocessing

- **Encoding categorical data**
- The data set may contain different types of data including continuous, and categorical data
- Some algorithms can work with categorical data directly. For example, a decision tree can be learned directly from categorical data with no data transform required (this depends on the specific implementation).
- Many machine learning algorithms cannot operate on label data directly. They require all input variables and output variables to be numeric.
- Some implementations of machine learning algorithms require all data to be numerical. For example, scikit-learn has this requirement.

Data preprocessing

- **Encoding categorical data**
- This means that categorical data must be converted to a numerical form. If the categorical variable is an output variable, you may also want to convert predictions by the model back into a categorical form in order to present them or use them in some application.
- Depending on whether categorical data is input or target, and nominal or ordinal, we should use different techniques to convert them into numerical values.

Data preprocessing

- **Encoding categorical target (output) data**
- We can use the LabelEncoder from the scikit-learn module to encode categorical target variables

Example:

```
from sklearn import preprocessing
le = preprocessing.LabelEncoder()
coded_target=le.fit_transform(["paris", "paris", "tokyo", "amsterdam"])
print(coded_target)
```

Data preprocessing

- **Encoding categorical input data – ordinal data**
- We can use the OrdinalEncoder from the scikit-learn module to encode ordinal input variables

Example:

```
import numpy as np
from sklearn.preprocessing import OrdinalEncoder
# define data
data = np.array([['red'], ['green'], ['blue']])
print(data)
# define ordinal encoding
encoder = OrdinalEncoder()
# transform data
result = encoder.fit_transform(data)
print(result)
```

Data preprocessing

- **Encoding categorical input data – ordinal data**
- We can use the OrdinalEncoder from the scikit-learn module to encode ordinal input variables

<u>Example (adult dataset):</u>

```
import numpy as np
from sklearn.preprocessing import OrdinalEncoder
encoder = OrdinalEncoder()
# transform data
data['education'] = encoder.fit_transform(data[‘education'].values.reshape(-1,1))
```

Data preprocessing

- **Encoding categorical input data – nominal data**
- We can use the OneHotEncoder from the scikit-learn module to encode nominal input variables

Example:

```
import numpy as np
from sklearn.preprocessing import OneHotEncoder
# define data
data = np.array([['red'], ['green'], ['blue']])
print(data)
# define ordinal encoding
encoder = OneHotEncoder(sparse=False)
# transform data
onehot = encoder.fit_transform(data)
print(onehot)
```

Data preprocessing

- **Encoding categorical input data – nominal data**

NOTE: For categorical variables where no ordinal relationship exists, the integer encoding may not be enough, at best, or misleading to the model at worst.

NOTE: Forcing an ordinal relationship via an ordinal encoding and allowing the model to assume a natural ordering between categories may result in poor performance or unexpected results (predictions halfway between categories).

NOTE: There is a problem with one hot encoding as its representation includes redundancy. For example, if we know that [1, 0, 0] represents "blue" and [0, 1, 0] represents "green" we don't need another binary variable to represent "red", instead we could use 0 values for both "blue" and "green" alone, e.g. [0, 0]. This is called a **dummy variable encoding**, and always represents C categories with C-1 binary variables.

Data preprocessing

- **Encoding categorical input data – nominal data – dummy variable**
- The "drop" argument in OneHotEncoder can be set to indicate which category will become the one that is assigned all zero values, called the "baseline". We can set this to "first" so that the first category is used. When the labels are sorted alphabetically, the first "blue" label will be the first and will become the baseline.

Example

```
import numpy as np
from sklearn.preprocessing import OneHotEncoder
# define data
data = np.array([['red'], ['green'], ['blue']])
print(data)
# define ordinal encoding
encoder = OneHotEncoder(drop='first', sparse=False)
# transform data
onehot = encoder.fit_transform(data)
print(onehot)
```

Data preprocessing

- **Data Normalisation**
- Normalisation refers to rescaling real-valued numeric attributes into the range 0 and 1.
- In machine learning algorithms, it is a good practice to normalize the columns (features) before training
- Normalisation also makes the training process less sensitive to the scale of the features
- It rarely happens that we need to normalize samples (rows)
- We have two common methods from the sklearn module to normalize data as follows: MinMaxScaler and normalize

Data preprocessing

- **Data Normalisation – normalize()**

- The normalize() method scales input vectors individually to the unit norm (vector length).
- The normalize() methods apply normalization to each row of the input data frame by default. To apply the normalization to each column you should set axis=0

Example

```
import pandas as pd
from sklearn.preprocessing import normalize
df = pd.read_csv('data.csv')
df=df.dropna()    ### test1: remove this to see what happpens
df_normalised=normalize(df.iloc[:,1:3]) ### test 3 use df.iloc[:,1] hint: don't forget to specify the axis
df_normalised=pd.DataFrame(df_normalised,columns=['a','b’]) ### test2 remove this
print(df_normalised)
```

Data preprocessing

- **Data Normalisation – MinMaxScaler()**
- The MinMaxScaler () transforms features by scaling each feature (column) to a given range using the following formula:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Example

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
pd.set_option('display.max_rows', 500)
pd.set_option('display.max_columns', 500)
df = pd.read_csv('data.csv')
df=df.dropna()    ### test1: remove this to see what happpens
scaler = MinMaxScaler()
df_normalised=scaler.fit_transform(df) # test 3 use df.iloc[:,1] hint: don't forget to specify the axis
df_normalised=pd.DataFrame(df_normalised,columns=['a','b','c','d']) ## test2 remove this
print(df_normalised)
```

Data preprocessing

- **Data Standardization – StandardScaler()**

- Standardization refers to shifting the distribution of each attribute to have a mean of zero and a standard deviation of one (unit variance).
- The standard score of an array x is calculated as z = (x - u) / s where u and s are the mean and standard deviation of x
- We can use sklearn StandardScaler() to standardize the columns of a data frame.

Example

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
pd.set_option('display.max_rows', 500)
pd.set_option('display.max_columns', 500)
df = pd.read_csv('data.csv')
df=df.dropna()    ### test1: remove this to see what happpens
scaler = StandardScaler()
df_normalised=scaler.fit_transform(df) # test 3 use df.iloc[:,1] hint: don't forget to specify the axis
df_normalised=pd.DataFrame(df_normalised,columns=['a','b','c','d']) ## test2 remove this
print(df_normalised)
```

Data preprocessing

- **Applying transformers to multiple columns**

- A transformer is a method that transforms the column, for example, normalisers and encoders are two types of transformers.
- The sklearn.compose module has a method ColumnTransformer that can be used to transform multiple columns using different types of transformers.
- All transformers should support fit() and transform() methods.

Syntax

Transformer_name= ColumnTransformer([("transfomer1_name", transformer1,[columns_name_or_indices]), ("transfomer2_name", transformer2,[columns_name_or_indices])], remainder="passthrough"/ "drop")
new_data_frame= Transformer_name.fit_transform(old_data_frame))

NOTE: If we set remainder="passthrough" then the remaining columns will only pass through without being affected. If remainder="drop" the remaining columns will be dropped from the table.

Data preprocessing

- **Applying transformers to multiple columns**

Example

Assume the training dataset "train_data" is a table with eight columns. Apply MinMaxScaler to columns 0 and 1, OneHotEncoder to columns 2 and 3, and OrdinalEncoder to columns 4 and 5.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder

ct= ColumnTransformer([("t1", MinMaxScaler(),[0,1]), ("t2", OneHotEncoder(),[2,3]), ("t3", OrdinalEncoder(),[4,5])],
remainder="passthrough")

New_train_data=ct.fit_transform(train_data)
```

Data preprocessing

- **Applying transformers to multiple columns**

Example

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import OneHotEncoder
ct = ColumnTransformer(
    [("t1", MinMaxScaler(), [0, 1]),("t2", OrdinalEncoder(), [2]),
     ("t3", OneHotEncoder(drop="first"), [3])],remainder='passthrough')
X = pd.DataFrame([[0., 2., "Saturday", "apple"],[2., 1., "Sunday", "banana"],[3., 4., "Monday", "orange"]])
print(X)
print(ct.fit_transform(X))
```